

## Software testing lifecycle

The **Software Testing Lifecycle (STLC)** refers to the sequence of distinct phases that testing teams follow to ensure the software product meets its intended quality standards. It outlines the systematic process of planning, designing, executing, and evaluating tests to identify defects and verify that the software behaves as expected. The STLC is a crucial part of the **Software Development Lifecycle (SDLC)**, but while SDLC focuses on developing the software, STLC focuses specifically on testing and ensuring the quality of that software.

The STLC typically consists of the following phases:

### 1. Requirement Analysis and Test Planning

#### 1.1 Requirement Analysis

**Objective:** The first phase of the STLC involves understanding the project requirements and identifying the testing needs.

**Key Activities:**

- Review the **functional** and **non-functional** requirements, such as performance, security, and usability, to understand the scope of testing.
- Identify testing requirements for various features, modules, and use cases.
- Assess the testability of the requirements (whether the requirements can be effectively tested).
- Define **test objectives** and scope for the project.
- **Outcome:** Test teams will develop a deeper understanding of the software being tested and its goals. This phase also identifies any gaps in requirements from a testing perspective.

#### 1.2 Test Planning

**Objective:** Once the requirements are clear, the test planning phase begins. This involves creating a comprehensive plan that outlines how testing will be performed.

**Key Activities:**

- Develop the **test strategy**: Define the overall approach for testing, such as manual vs. automated testing, risk-based testing, and resource allocation.
- Create a **test plan document** that includes:

- Scope of testing (which features or modules will be tested).
- Types of testing to be performed (functional, performance, security, etc.).
- Testing tools and technologies to be used.
- Resource planning (team, environment, test data).
- Timeline and milestones.
- Risks and mitigation strategies.
- Define entry and exit criteria for each phase.
- **Outcome:** A clear, structured test plan that guides all subsequent testing activities.

## 2. Test Design

### • 2.1 Test Case Design

**Objective:** In this phase, detailed test cases are created based on the requirements and the test plan. Test cases define the specific conditions under which a particular part of the software will be tested.

**Key Activities:**

- **Test case creation:** Each test case includes:
  - Test case ID.
  - Test description.
  - Input data.
  - Expected results.
  - Test execution steps.
  - Pass/fail criteria.
- **Test data preparation:** Create or gather data that will be used during testing, ensuring it covers valid, invalid, and edge case scenarios.
- **Test environment setup:** Set up the testing environment, which may include hardware, software, network configurations, and databases.

**Outcome:** A detailed set of test cases, test scripts (for automated testing), and test data ready for execution. This phase ensures that testers have a clear understanding of how each feature or functionality will be tested.

## 2.2 Test Design Review

**Objective:** A review of the test cases and test environment to ensure that all aspects of the software are covered and no critical test scenarios are missed.

**Key Activities:**

- Review the test cases with stakeholders (test leads, developers, business analysts) to ensure completeness and correctness.
- Modify test cases based on feedback.
- Finalize the test design and prepare for execution.

**Outcome:** Finalized, reviewed test cases and test scripts that are aligned with the requirements.

## 3. Test Execution

### 3.1 Test Execution

**Objective:** During this phase, the actual testing of the software takes place by running the test cases as defined in the test design phase.

**Key Activities:**

- **Executing test cases:** Testers manually or automatically execute test cases on the software, either in isolation or as part of the integrated system.
- **Logging defects:** If any issues or defects are identified, they are logged in a defect tracking tool (e.g., JIRA, Bugzilla) with detailed information, including steps to reproduce, severity, and environment.
- **Test results:** Record the test results (pass/fail) and document any deviations from the expected behavior.

**Outcome:** Test results with a list of executed test cases, passed/failed results, and identified defects.

### 3.2 Defect Reporting and Tracking

**Objective:** After executing the tests, any defects found must be reported, tracked, and managed.

• **Key Activities:**

- Report the defects with detailed information (description, severity, steps to reproduce).

- Track defect resolution progress, from the time they are logged until they are fixed and retested.
  - Communicate defects to the development team for investigation and resolution.
- **Outcome:** A defect log that includes details of each identified defect and the status of their resolution.

## 4. Defect Management and Retesting

### 4.1 Defect Fixing

- **Objective:** Once the defects are reported, the development team works to fix the issues identified during testing.
- **Key Activities:**
  - Developers investigate the defects, make necessary changes to the code, and provide fixes.
  - Developers may reassign fixed defects to the test team for retesting.
- **Outcome:** A list of defects resolved, which will then be validated by the testing team.

### 4.2 Retesting and Regression Testing

- **Objective:** After the defects have been fixed, the software must be retested to verify that the fixes work and that no new defects have been introduced.
- **Key Activities:**
  - **Retesting:** Execute the same test cases that initially failed to verify that the defects have been fixed.
  - **Regression Testing:** Verify that the new changes (e.g., defect fixes, new features) haven't negatively impacted other parts of the system that were previously working.
  - Ensure that the software is stable and ready for final release.
- **Outcome:** Confirmation that defects are fixed and the software continues to function as expected after changes.

## 5. Test Closure

### 5.1 Test Closure Activities

- **Objective:** This is the final phase of the STLC, where testing activities are formally concluded, and the test results are documented.
- **Key Activities:**
  - **Test summary report:** Prepare a test summary report that includes:
    - Number of test cases executed.



- Number of defects found and their status (resolved, pending).
  - Test coverage.
  - Testing activities and their outcomes.
  - Lessons learned and suggestions for future projects.
- **Test artifacts archiving:** All test documentation, including test cases, defect logs, and test results, should be stored and archived for future reference, audits, or post-release analysis.
- **Retrospective:** Review the entire testing process with stakeholders to discuss challenges, successes, and areas for improvement in future testing cycles.
- **Outcome:** A formal sign-off on testing, along with comprehensive test documentation and a record of test results. The project is ready for release or deployment.

## 6. Post-Release and Maintenance Testing

### 6.1 Post-Release Testing

- **Objective:** After the software has been released, the testing team may conduct post-release testing to ensure that the software performs as expected in a real-world environment.
- **Key Activities:**
  - Monitor the software in production to detect any defects or performance issues that were not caught during earlier testing phases.
  - Perform **maintenance testing** as needed to ensure that fixes or enhancements do not impact the existing functionality.
  - Provide support for any emergency patches or updates required in production.
- **Outcome:** Ongoing testing and quality assurance to maintain the software's quality after release.

The **Software Testing Lifecycle (STLC)** is a structured and methodical approach to ensuring software quality. It begins with understanding the requirements and planning, continues through test design, execution, and defect management, and ends with test closure and post-release activities. Each phase builds upon the previous one to ensure that defects are identified and fixed in a timely manner, and that the software meets both functional and non-functional requirements. Properly

following the STLC helps teams deliver reliable, high-quality software while managing risks and ensuring continuous improvement.



# PUACP